

Software Design Specifications

Inventory Management for Small Businesses

Version 1.0

Prepared by: Team-23

Primary backend contributor: Sohan Varma - SE23UCSE163

Academic context: Software Engineering Course, Mahindra University

Document date: 09-Feb-2026

Document Information

Title	Inventory Management for Small Businesses - Software Design Specification	Project Manager	Team-23 / Course Project Team
Document Version No.	1.0	Document Version Date	09-Feb-2026
Prepared By	Team-23	Preparation Date	09-Feb-2026
Primary Author	Sohan Varma	Position	Backend Developer
Reviewed By	Project Team	Status	Final Draft

Version History

Ver. No.	Ver. Date	Revised By	Description	Filename
1.0	09-Feb-2026	Team-23	Initial SDS prepared for the project	inventory_management_sds.docx

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
- 2. Use Case View
 - 2.1 Use Cases
- 3. Design Overview
 - 3.1 Design Goals and Constraints
 - 3.2 Design Assumptions
 - 3.3 Significant Design Packages
 - 3.4 Dependent External Interfaces
 - 3.5 Implemented Application External Interfaces

4. Logical View

4.1 Design Model

4.2 Use Case Realization

5. Data View

5.1 Domain Model

5.2 Data Model (Persistent Data View)

5.2.1 Data Dictionary

6. Exception Handling

7. Configurable Parameters

8. Quality of Service

8.1 Availability

8.2 Security and Authorization

8.3 Load and Performance Implications

8.4 Monitoring and Control

1 Introduction

This Software Design Specification (SDS) describes the architecture, module structure, data model, interfaces, and quality considerations for the Inventory Management for Small Businesses project. The system is designed to help small businesses manage products, stock levels, sales, purchases, suppliers, and branch-wise inventory while supporting operational visibility through reports and dashboard analytics.

1.1 Purpose

The purpose of this document is to translate the project requirements into a concrete software design that can guide development, testing, review, and demonstration. The intended audience includes developers, database designers, testers, system reviewers, and faculty evaluators.

1.2 Scope

This specification applies to the web-based Inventory Management system developed for small business operations. The design covers user authentication, role-based access, product and category management, stock-in and stock-out tracking, sales processing, supplier management, sales and purchase history, low-stock alerts, branch-wise stock tracking, stock transfer support, and reporting.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface used for communication between frontend and backend.
JWT	JSON Web Token used to identify an authenticated session in a stateless way.
RBAC	Role-Based Access Control that limits actions based on user role, such as Admin or Staff.
SKU	Stock Keeping Unit used to uniquely identify a product.
Low-stock alert	A system-generated signal when available quantity falls below a configured threshold.
Branch	A business location whose inventory may be managed independently.

1.4 References

- Project Software Design Specification template provided by the user.
- Statement of Work (Team-23), dated 09-Feb-2026.
- UML diagrams prepared for the Inventory Management System (class, activity, component, and sequence views).
- Login interface and implementation discussions covering Admin and Staff access control.

2 Use Case View

The Inventory Management system supports day-to-day inventory operations for small business users. The central actors are Admin and Staff. Admin has full operational access, while Staff is limited to day-to-day inventory and sales activities.

2.1 Use Cases

UC-01 Login

Primary Actor: Admin / Staff

Description: Authenticate and enter the system based on role permissions.

Main Flow:

1. Open the login page.
2. Enter valid username and password.
3. System validates credentials and creates session/token.
4. User is redirected to the dashboard allowed for that role.

UC-02 Manage Products

Primary Actor: Admin

Description: Create, update, and organize products and categories.

Main Flow:

5. Admin opens product management.
6. Adds or edits name, category, SKU, price, and branch mapping.
7. System validates fields and stores the record.
8. Updated product list is displayed.

UC-03 Record Sale

Primary Actor: Staff / Admin

Description: Sell an item and reduce available stock in real time.

Main Flow:

9. User selects product and quantity.
10. System checks current stock.
11. If quantity is sufficient, the sale is stored and stock is reduced.
12. Confirmation is shown; otherwise an out-of-stock error is returned.

UC-04 Manage Suppliers and Purchases

Primary Actor: Admin

Description: Maintain supplier records and incoming stock transactions.

Main Flow:

13. Admin creates or updates supplier details.

14. Purchase or stock-in transaction is recorded.
15. Inventory quantity is increased.
16. History is available for audit and review.

UC-05 Low-Stock Monitoring

Primary Actor: Admin / Staff

Description: Identify items nearing shortage.

Main Flow:

17. System compares current quantity against threshold.
18. If quantity is low, an alert is generated.
19. Users view alert in dashboard or stock screens.
20. Admin can plan replenishment.

UC-06 Branch Tracking / Transfer

Primary Actor: Admin

Description: Track stock separately by branch and transfer stock when required.

Main Flow:

21. Admin selects source and target branch.
22. System checks availability in source branch.
23. Transfer is recorded and branch quantities are updated.
24. Audit trail is maintained.

UC-07 Reporting and Analytics

Primary Actor: Admin

Description: View reports on sales, stock, and trends.

Main Flow:

25. Admin opens reporting dashboard.
26. System retrieves sales history, inventory summaries, and trend data.
27. Charts and tables are displayed.
28. Admin uses outputs for business decisions.

3 Design Overview

The system follows a layered web application design. The presentation layer handles pages, forms, and dashboards. The API/controller layer receives HTTP requests and validates user input. The service layer contains business logic for inventory, sales, suppliers, and reporting. The data layer persists entities such as users, products, sales, orders, and suppliers in a relational database. Authentication and role checks are applied before protected operations are executed.

3.1 Design Goals and Constraints

- Provide a simple and usable interface suitable for small business workflows.
- Maintain accurate real-time stock levels after each stock-in, sale, or transfer.
- Support Admin and Staff roles with controlled permissions.
- Preserve sales and purchase history for traceability.
- Keep the design modular so product, supplier, reporting, and branch features remain easy to extend.
- Meet academic project deadlines and work within limited development time and team bandwidth.

3.2 Design Assumptions

- Users access the application through a browser on a local or campus network environment.
- Each product has a unique identifier and can be associated with inventory quantities.
- The backend validates all stock-changing operations before committing them.
- Only authenticated users may access protected pages or API routes.
- Analytics are derived from operational data already stored by the application.

3.3 Significant Design Packages

Package / Module	Responsibility
UI Layer	Login page, dashboard, product pages, supplier pages, sales screens, and reporting views.
Authentication Module	Credential validation, token/session handling, route protection, and role checks.
Inventory Module	Product catalog, stock validation, low-stock rules, and branch-level quantity management.
Sales Module	Sale creation, stock deduction, sales history, and transaction confirmation.
Supplier / Purchase Module	Supplier master data, incoming stock entries, and reorder support.
Reporting Module	Sales reports, stock reports, trend summaries, and dashboard metrics.
Persistence Layer	Database schema, CRUD operations, and query execution.

3.4 Dependent External Interfaces

External Application / Interface	Module Using the Interface	Functionality / Description
Browser UI	Frontend pages	Used by Admin and Staff to interact with login, inventory, supplier, sales, and reporting screens.

Barcode / camera input (optional integration point)	Inventory / product lookup module	Can be used to identify products quickly during stock updates or sales.
Relational database	Backend service and persistence layers	Stores users, products, suppliers, sales, purchases, and stock records.

3.5 Implemented Application External Interfaces

Interface Name	Module Implementing the Interface	Functionality / Description
Authentication API	Auth controller / auth service	Validates user credentials and returns authenticated session access.
Product API	Inventory service	Supports create, read, update, and stock lookup operations for products.
Sales API	Sales controller / inventory service	Validates stock, records sales, deducts quantity, and returns success or out-of-stock errors.
Supplier API	Supplier service	Creates, edits, and fetches supplier information.
Reporting API	Reporting service	Returns stock and sales summaries used by dashboards and reports.

4 Logical View

The logical view organizes the software into cooperating modules. At runtime, user actions begin at the frontend, travel through controllers and services, and finally reach the database. Validation and authorization are enforced before stock or sales data is changed.

4.1 Design Model

The design model is centered around the following significant classes or entities: User, Product, Inventory, Sale, Order, Supplier, and Report. User is responsible for authentication context and role information. Product stores item identity, pricing, and descriptive attributes. Inventory maintains current stock and thresholds. Sale represents a stock-reducing transaction. Order and Supplier support replenishment. Report aggregates operational data into summaries for decision support.

Relationships are straightforward: users manage products and create sales; products belong to inventory; sales reduce inventory; orders are fulfilled by suppliers; reports are generated from sales and inventory data.

4.2 Use Case Realization

Record Sale is the most important end-to-end transaction. The user enters product and quantity in the sales page. The request is sent to the sales API. The inventory service validates that the quantity is available. If stock is sufficient, product quantity is updated and the sales record is inserted in the database. The service then returns a success response and the UI displays confirmation. If stock is insufficient, the service returns an error and no inventory update is committed.

Login is realized through the authentication module. The user submits credentials, the backend verifies them, and a protected session or token is created. Subsequent access to protected routes is checked by authentication middleware, and role-based guards determine which operations are visible or allowed.

5 Data View

The application uses persistent relational storage so that stock levels, transactions, user accounts, and supplier records remain consistent across sessions. The data design emphasizes traceability and straightforward querying.

5.1 Domain Model

The core domain entities are User, Product, Inventory, Sale, Purchase/Order, Supplier, Branch, and Report. A User performs actions based on role. A Product is stocked in inventory and may exist across one or more branches. A Sale reduces stock. A Purchase or replenishment increases stock. Suppliers are linked to product procurement. Reports summarize data from operational entities.

5.2 Data Model (Persistent Data View)

A normalized relational model is suitable for the project. Products, suppliers, and users are maintained as master tables. Sales, purchases, and transfers are transaction tables. Inventory quantities may be stored directly in a stock table or derived from transaction history depending on implementation choice.

5.2.1 Data Dictionary

Table / Entity	Key Fields	Purpose	Notes
Users	user_id, username, password_hash, role	Stores login and role information.	Roles include Admin and Staff.
Products	product_id, sku, name, category_id, price	Stores product master data.	Referenced by inventory, sales, and purchases.
Inventory	inventory_id, product_id, branch_id, quantity, low_stock_limit	Stores current stock by product and optionally by branch.	Supports low-stock alerts.

Suppliers	supplier_id, supplier_name, contact, phone	Stores supplier details.	Used for procurement and replenishment.
Sales	sale_id, product_id, quantity, total_amount, sale_date, created_by	Stores each completed sale transaction.	Reduces available stock.
Purchases	purchase_id, supplier_id, product_id, quantity, purchase_date	Stores incoming stock transactions.	Increases available stock.
Branches	branch_id, branch_name, location	Stores branch metadata.	Used when branch-wise tracking is enabled.
Stock_Transfers	transfer_id, product_id, source_branch_id, target_branch_id, quantity, transfer_date	Tracks movement between branches.	Provides accountability and audit trail.

6 Exception Handling

The system must handle validation and operational failures gracefully. Typical exceptions include invalid login, duplicate product identifiers, negative quantity input, out-of-stock sales, failed supplier updates, and invalid branch transfer requests. Backend services should return structured error messages to the UI. Important failures should be logged with enough context to reproduce the issue, while sensitive values such as passwords must never be logged.

7 Configurable Parameters

The following parameters are configurable and may be stored in environment variables, configuration files, or system settings.

Configuration Parameter Name	Definition and Usage	Dynamic?
LOW_STOCK_DEFAULT	Default threshold used to trigger a low-stock alert.	Yes
JWT_SECRET / SESSION_SECRET	Secret used to sign authentication tokens or sessions.	No
TOKEN_EXPIRY	Controls how long an authenticated token remains	Yes

	valid.	
REPORT_RANGE_DEFAULT	Default date range for dashboard and report views.	Yes
SERVER_PORT	Application runtime port.	No
DB_CONNECTION_STRING	Database connection configuration.	No

8 Quality of Service

8.1 Availability

Because the application supports operational inventory work, availability matters during business hours. The design should minimize downtime caused by deployment or maintenance. Database backups and basic recovery procedures are recommended so transaction history is not lost.

8.2 Security and Authorization

Security is enforced through authenticated login, protected routes, and role-based authorization. Admin users can access setup and master-data operations, while Staff users are restricted to permitted day-to-day functions such as sales entry and stock viewing. Passwords must be stored securely as hashes, and tokens or sessions must be validated for every protected request.

8.3 Load and Performance Implications

The application is intended for a small business environment, so the expected load is moderate. However, stock updates and sales recording must remain responsive because they occur in real time. Read-heavy operations such as dashboard summaries and reporting should use efficient indexed queries. Transaction flows that update stock should be atomic so inventory quantities remain consistent under concurrent access.

8.4 Monitoring and Control

The backend should log authentication attempts, API errors, and important inventory-changing events. Basic health checks for server startup, database connectivity, and API availability help during testing and demonstration. Audit fields such as `created_by`, `updated_by`, and timestamps improve traceability.

End of Document